TatvaSoft

SOFTWARE DEVELOPMENT · GUIDELINE

TVS-DEV-AI-GL-01.8

MCP Integration with AI Editors & Tools

MCP for Developers — Awareness, Benefits, Integration & Safe Usage

VERSION	EFFECTIVE DATE	PREPARED BY	APPROVED BY
1.0	28.09.2026	Vishal Shah	Shital Patel

HOW TO READ THIS DOCUMENT

Short on time? Read the **Key Takeaway** and the **Security & Permission Guidelines**.

The **Figma** and **Jira** examples show a typical setup and workflow, step by step.

Parent guideline: **TVS-DEV-AI-GL-01 AI Development Tools Guidelines** (Model Context Protocol (MCP) section).

Document Control

Document Control Information

Document Title	MCP Integration with AI Editors & Tools
Document ID	TVS-DEV-AI-GL-01.8
Document Version	Version 1.0
Effective Date	28.09.2026
Parent Document	TVS-DEV-AI-GL-01 — AI Development Tools Guidelines

Document Publication History

Document Prepared By	Vishal Shah
Document Reviewed By	Jigar Prajapati
Document Approved By	Shital Patel
Initial Release Version	1.0
Initial Release Date	28.09.2026

Document Revision History

Version No.	Revision Description	Change Author	Change Approver	Revision Date
1.0	Final Release	Vishal Shah	Shital Patel	25.09.2026

Document Definition

Model Context Protocol (MCP) is a standard way for AI applications to connect with external tools, systems and data sources. Instead of keeping the AI editor isolated from the tools developers use every day, MCP allows the editor to interact with approved systems through an MCP server.

Contents

What is MCP and Why is it Used?	How MCP connects the AI editor to external tools
Benefits of MCP for Developers	Why developers use MCP
Security & Permission Guidelines	Least privilege and safe usage
Example: Integrating Figma MCP	Typical setup and workflow
Example: Integrating Jira MCP	Typical setup and workflow

What is MCP and Why is it Used?

> **KEY TAKEAWAY**
>
> **Model Context Protocol (MCP)** is a standard way for AI applications to connect with external tools, systems and data sources. Instead of keeping the AI editor isolated from the tools developers use every day, MCP allows the editor to interact with **approved systems** through an **MCP server**.

For example, an AI editor can use a **Figma MCP server** to understand a design and help generate code based on it, or use a **Jira MCP server** to read project issues and update information when the required permissions are available.

A typical flow looks like this:



*The AI editor sends a request to the MCP server, and the MCP server performs the **permitted operation** against the connected system.*

MCP is useful because developers can work with external systems **without repeatedly switching** between the AI editor and other applications. It can also give the AI **more relevant project context** and allow repetitive tasks to be handled through **natural-language instructions**.

Benefits of MCP for Developers

- ✓ Access information from external tools directly from the AI editor.
- ✓ Reduce context switching between the code editor and tools such as Figma, Jira or documentation systems.
- ✓ Give the AI better context by allowing it to retrieve relevant project information.
- ✓ Automate repetitive tasks that would otherwise require manual navigation.
- ✓ Use natural-language instructions for supported operations instead of manually performing every step.
- ✓ Keep development work connected with the actual project information and design requirements.
- ✓ Create opportunities to connect AI editors with internal tools and approved company systems.
- ✓ Improve developer productivity when the MCP integration is configured with appropriate permissions and clear usage guidelines.

Security & Permission Guidelines

MCP gives an AI editor a way to interact with other systems. The **permissions granted to the MCP connection** therefore matter as much as the MCP integration itself.

> **PRINCIPLE OF LEAST PRIVILEGE**
> Developers should follow the **principle of least privilege**: provide only the access that is actually required for the task.

#	Guideline
1	**Use the minimum permissions required for the MCP connection.** If the developer only needs to retrieve information, use a Figma/Jira account or authentication token with read-only access. Avoid granting edit, delete, or administrative permissions unless they are specifically required.
2	**Restrict unnecessary MCP tools in the AI editor.** If the connected Figma/Jira account already has edit or delete permissions, remove or disable the corresponding destructive tools from the AI editor/MCP configuration where supported. For example, if Jira `delete_issue` is not required, it should not be made available to the developer through the AI editor.
3	**Use separate development/test credentials** where practical instead of connecting an AI tool directly to sensitive production accounts.
4	**Never** place API keys, access tokens or passwords directly in source code or commit them to Git.
5	**Do not** paste sensitive credentials into AI prompts or chat conversations.
6	**Understand what data the MCP server can access** before connecting it to an account.
7	**Use MCP servers from trusted and approved sources.** Avoid installing unknown MCP servers simply because they appear useful.
8	**Review the permissions** of an MCP integration when it is installed or updated.
9	**Disable or remove** MCP connections that are no longer required.
10	Before allowing a write operation, **make sure the AI is acting on the correct project, file, issue or environment.**
11	For destructive or high-impact operations, **review the intended action** before allowing it to execute.

> **RECOMMENDED APPROACH**
> Start with the **minimum required permissions**. Increase access only when there is a clear business or development need.

Example: Integrating Figma MCP

Figma MCP can be used to make design information available to an AI editor. A common development scenario is to select or provide a Figma design and ask the AI editor to understand the design and generate or update the corresponding frontend code.

A typical setup consists of:

- ☐ A Figma account with access to the required design files.
- ☐ An AI editor that supports MCP.
- ☐ A Figma MCP server configured in the AI editor.
- ☐ Authentication that allows the MCP server to access the required Figma resources.
- ☐ A development project where the generated or updated code can be reviewed.

Typical workflow:

1. Configure the approved Figma MCP server in the AI editor.
2. Authenticate using the appropriate Figma account or access method.
3. Open the relevant Figma design or provide the required design reference.
4. Ask the AI editor to inspect the design and explain or implement the required UI.
5. Review the generated code and compare it with the Figma design.
6. Make any required adjustments and test the implementation.

> **EXAMPLE VIDEO**
>
> To generate the Figma token and configure the Figma MCP with Cursor AI.
>
> https://upskill.tatvasoft.com/Cursor/U2FsdGVkX19eIOBD68r49hmCFxBjflGyEEonsTdTi8c

Example: Integrating Jira MCP

Jira MCP can allow an AI editor to retrieve project and issue information and, depending on the permissions provided, create or update Jira issues. This can be useful when developers need issue context while working on code.

A typical setup consists of:

- ☐ An approved Jira MCP server.
- ☐ An AI editor with MCP support.
- ☐ A Jira account or approved authentication method.
- ☐ Access to the required Jira project(s).
- ☐ A permission model that limits the actions the MCP connection can perform.

Example workflow:

1. Configure the approved Jira MCP server in the AI editor.
2. Authenticate using the appropriate Jira account.
3. Ask the AI to retrieve a specific Jira issue or project requirement.
4. Use the retrieved information as context while implementing the task.
5. If Jira updates are required, clearly specify the **intended issue and fields** to be changed.
6. **Review the update** before allowing the AI to execute it.

— End of Document —